Architecting Network Automation

Egor Krivosheev 2degrees

Management Plane	NETCONF RESTCONF gRPC
Control Plane	Proprietary APIs
Forwarding Plane	P4

gRPC

- gRPC - is an open source remote procedure call system
- Uses HTTP/2 for transport, Bi-directional streaming
- Protocol Buffers as the interface description language
- Two interfaces: Open Config Telemetry and gNMI

NETCONF

- RFC6241
- XML encoding
- SSH as transport
- There are libraries for many languages Python, Go, C, Java, PHP

RESTCONF

- RFC8040
- XML or JSON as encoding
- Configuration management
- Operational data
- HTTP POST, GET, PUT, DEL to manage configuration or collect operational data

RESTCONF VS NETCONF

In RESTCONF you need one request to change an interface config another one to change BGP and so on. No verification, instead each request succeeds or fails

In NETCONF you lock the config make all the changes, verify, then commit and unlock

Control Plane - Proprietary APIs

- Juniper JET and Cisco IOS-XR Service Layer both gRPC based
- Provide direct access to things like firewall policies or CoS without a need for a configuration change
- Much faster than dealing with management plane
- Cisco: BFD, Interfaces, MPLS, Routing
- Juniper: Interfaces, MPLS, Routing, BGP, CoS and Firewall

Use cases

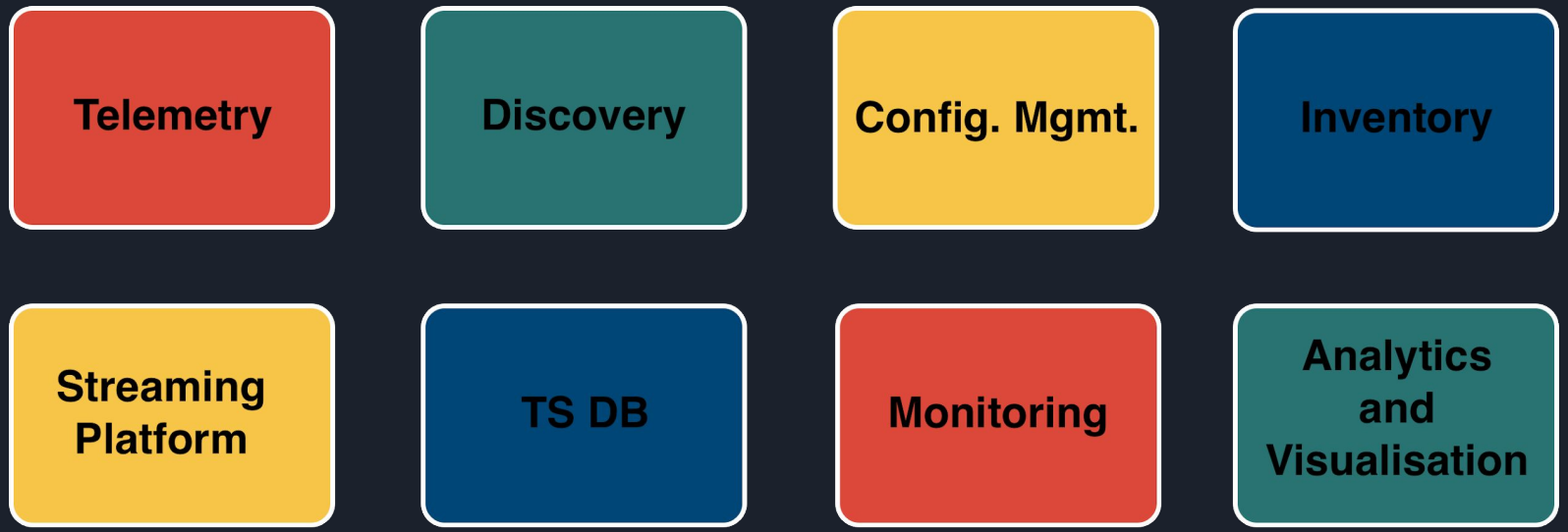
- Adjusting CoS policy on a microwave link
- Adding a lot of static routes in BGP for testing
- Quickly create an IP interface for automatic testing

Forwarding plane

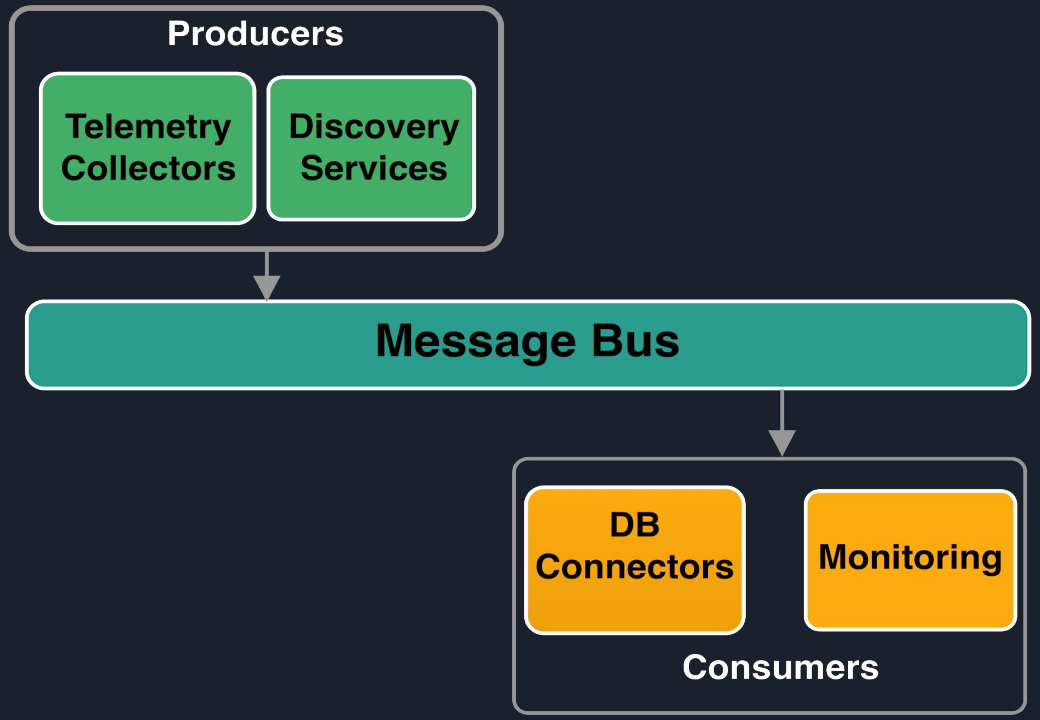
- With P4 programing language
- P4 Runtime supported by at least Arista, Juniper
- Available on whitebox switches running Barefoot Tofino ASICs
- Capabilities depend on hardware

What is P4?

- A domain specific language
- Open-source
- Protocol independent, Target independent
- Match-action pipelines, packet forwarding can be split into series of table lookups and corresponding header manipulations


Telemetry
Discovery
Config. Mgmt.
Inventory
Streaming Platform
TS DB
Monitoring
Analytics and Visualisation

Message Bus


Producers
Telemetry Collectors
Discovery Services
Message Bus
DB Connectors
Monitoring
Consumers

Inventory and Discovery

Inventory is a DB with APIs and a web interface

Discovery service is a program or a script which detects changes in hardware or software and sends updates into the message bus

Configuration management

- Is what people think when you mention network automation
- I think a combination of tools like Ansible or SaltStack and in-house written code works the best

Telemetry

- Protocol Buffers as encoding
- Collectors get data using gRPC, NETCONF, SNMP
- And publish/produce the data into the message bus
- Other systems subscribe/consume the data from the message bus

Having metrics allows to

- Check state to verify configuration changes
- React to network events
- Provide a feedback loop for various systems

Time Series DB

- Historically most of the metrics were stored in RRD files
- The amount of data has dramatically increased over years
- We now have specialized time series databases
- I use influx DB and Elasticsearch

Analytics and Visualisation

- No much use in data if you do not have tools to represent and analyse it
- The data analytics is needed for capacity planning and fault management
- The automation is needed here as well
- I use Grafana for visualisation and custom code to automate dashboard management

Exploring the idea of ChatOps

- Chat bots as universal interface for various tools
- To fetch a graph from Grafana or remove a faulty link from service

Why Go?

- Strongly Typed
- Statically Linked
- Great concurrency primitives out of the box
- Cross compiled
- Built-in race detector

Questions ?

Useful links:

- Streaming Telemetry how to with code example
- NETCONF how to with code example
- Kafka
- Elasticsearch
- Asnible
- SaltStack
- SSH Screen-scraping with Go
- NZNOG 2019 Ansible repository with examples
- Juniper Control Plane API
- Cisco Control Plane API